Unit 8: Debugging PHP

php.ini Settings

php.ini is a configuration file used by PHP, a popular server-side scripting language, to customize various settings and options. Here are some of the most common php.ini settings:

1) error_reporting: Determines the level of error reporting that PHP will display. You can set it to show all errors or just certain types of errors.

2) display_errors: Determines whether errors should be displayed in the output or not.

3) max_execution_time: Determines the maximum time that PHP scripts can run. This is useful for preventing scripts from running indefinitely and consuming too many server resources.

4) memory_limit: Determines the maximum amount of memory that PHP scripts can use. This is useful for preventing scripts from consuming too much memory and crashing the server.

5) upload_max_filesize: Determines the maximum size of uploaded files that PHP will accept.

6) post_max_size: Determines the maximum size of POST data that PHP will accept.

7) date.timezone: Sets the default timezone used by PHP functions.

8) extension_dir: Specifies the directory where PHP extensions are stored.

9) session.save_path: Specifies the location where session data is stored.

10) allow_url_fopen: Determines whether PHP is allowed to open URLs as if they were regular files.

These are just a few examples of the many settings that can be configured in php.ini. It's important to carefully consider and configure these settings to ensure that your PHP applications run smoothly and securely.

Here are some commonly used php.ini settings along with examples of how they can be used:

1.error_reporting: This setting determines which types of errors PHP will report. To report all errors, you can set it to E_ALL. For example:

error_reporting = E_ALL

2.display_errors: This setting determines whether errors should be displayed in the output or not. To display errors, you can set it to On. For example:

display_errors = On

3.max_execution_time: This setting determines the maximum time that PHP scripts can run. To set a limit of 30 seconds, for example, you can set it to 30. For example:

max_execution_time = 30

4.memory_limit: This setting determines the maximum amount of memory that PHP scripts can use. To set a limit of 128 megabytes, for example, you can set it to 128M. For example:

memory_limit = 128M

5.upload_max_filesize: This setting determines the maximum size of uploaded files that PHP will accept. To set a limit of 10 megabytes, for example, you can set it to 10M. For example:

upload_max_filesize = 10M

6.post_max_size: This setting determines the maximum size of POST data that PHP will accept. To set a limit of 20 megabytes, for example, you can set it to 20M. For example:

post_max_size = 20M

7.date.timezone: This setting sets the default timezone used by PHP functions. To set the timezone to US/Eastern, for example, you can set it to America/New_York. For example:

date.timezone = America/New_York

8.extension_dir: This setting specifies the directory where PHP extensions are stored. To set the directory to /usr/lib/php/extensions, for example, you can set it to /usr/lib/php/extensions. For example:

extension_dir = /usr/lib/php/extensions

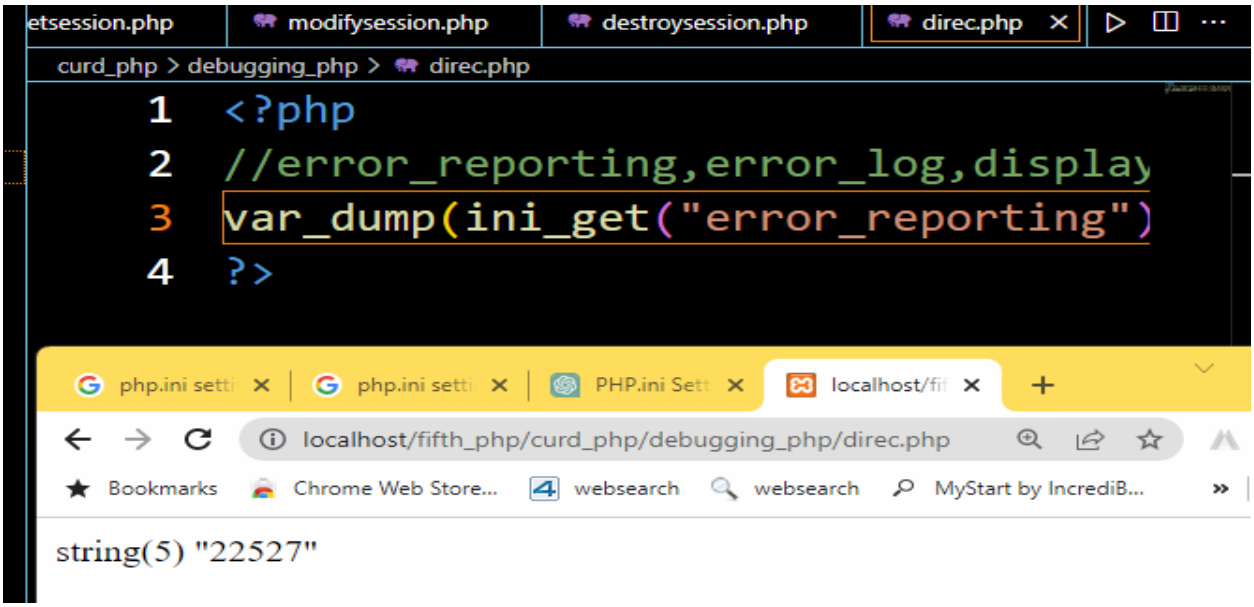
9.session.save_path: This setting specifies the location where session data is stored. To set the location to /var/lib/php/sessions, for example, you can set it to /var/lib/php/sessions. For example:

session.save_path = /var/lib/php/sessions

10.allow_url_fopen: This setting determines whether PHP is allowed to open URLs as if they were regular files. To enable this feature, you can set it to On. For example:

allow_url_fopen = On

practical on php_ini



The **var_dump(ini_get("error_reporting"))** statement will output the current value of the error_reporting configuration setting in PHP, which determines which types of errors and warnings are displayed.

The ini_get() function retrieves the current value of a configuration setting, and in this case, the setting being queried is error_reporting.

The output of the var_dump() function will depend on the current value of error_reporting. It will display the value as an integer, which can be interpreted using the following constants defined by PHP:

E_ALL - All errors and warnings
E_ERROR - Fatal run-time errors
E_WARNING - Run-time warnings
E_NOTICE - Notices indicating non-fatal errors
E_DEPRECATED - Notices indicating usage of deprecated features
E_STRICT - Run-time notices indicating coding standards issues
For example, if the output of var_dump(ini_get("error_reporting")) is int(22527), this means that the error_reporting setting is currently set to display all errors, warnings, and notices except for strict standards and deprecation notices.

string(5) "22527"
The output string(5) "22527" represents a string with 5 characters, which is the value of the error_reporting configuration setting in PHP.

The var_dump() function is used to display the data type and value of a variable, and in this case, it is displaying the data type as string and the value as "22527". This means that the error_reporting setting is currently set to a string value of "22527".

```php
<?php
//error_reporting,error_log,display_error
var_dump(ini_get('error_reporting'));
var_dump(E_ALL);
$array=[1];
echo $array[3];
```

localhost/fifth_php/curd_php/debugging_php/direc.php

string(5) "22527" int(32767)
Warning: Undefined array key 3 in
C:\xampp\htdocs\fifth_php\curd_php\debugging_php\direc.php on line **6**

curd_php > debugging_php > direc.php > ...

```php
<?php
//error_reporting,error_log,display_error
var_dump(ini_get('error_reporting'));
var_dump(E_ALL);

ini_set('error_reporting',E_ALL & ~E_WARNING);
$array=[1];
echo $array[3];


?>
```

localhost/fifth_php/curd_php/debugging_php/direc.php

string(5) "22527" int(32767)

string(5) "22527" int(32767)
Warning: Undefined array key 3 in
C:\xampp\htdocs\fifth_php\curd_php\debugging_php\direc.php on line **8**

d_php > debugging_php > direc.php > ...

```php
<?php
//error_reporting,error_log,display_error
var_dump(ini_get('error_reporting'));
var_dump(E_ALL);
ini_set('display_errors',0);

//ini_set('error_reporting',E_ALL & ~E_WARNING)
$array=[1];
echo $array[3];


?>
```

localhost/fifth_php/curd_php/debugging_php/direc.php

string(5) "22527" int(32767)

curd_php > debugging_php > direc.php

```php
<?php
//error_reporting,error_log,display_error
var_dump(ini_get('display_errors'));

//ini_set('max_execution_time',3);
//sleep(5);
//echo 'hello subash';
?>
```

localhost/fifth_php/curd_php/debugging_php/direc.php

string(1) "1"

curd_php > debugging_php > direc.php

```php
<?php
//error_reporting,error_log,display_error
var_dump(ini_get('display_errors'));

ini_set('max_execution_time',3);
sleep(5);
echo 'hello subash';
?>
```

localhost/fifth_php/curd_php/debugging_php/direc.php

string(1) "1"
Fatal error: Maximum execution time of 3 seconds exceeded in **C:\xampp\htdocs\fifth_php\curd_php\debugging_php\direc.php** on line 7

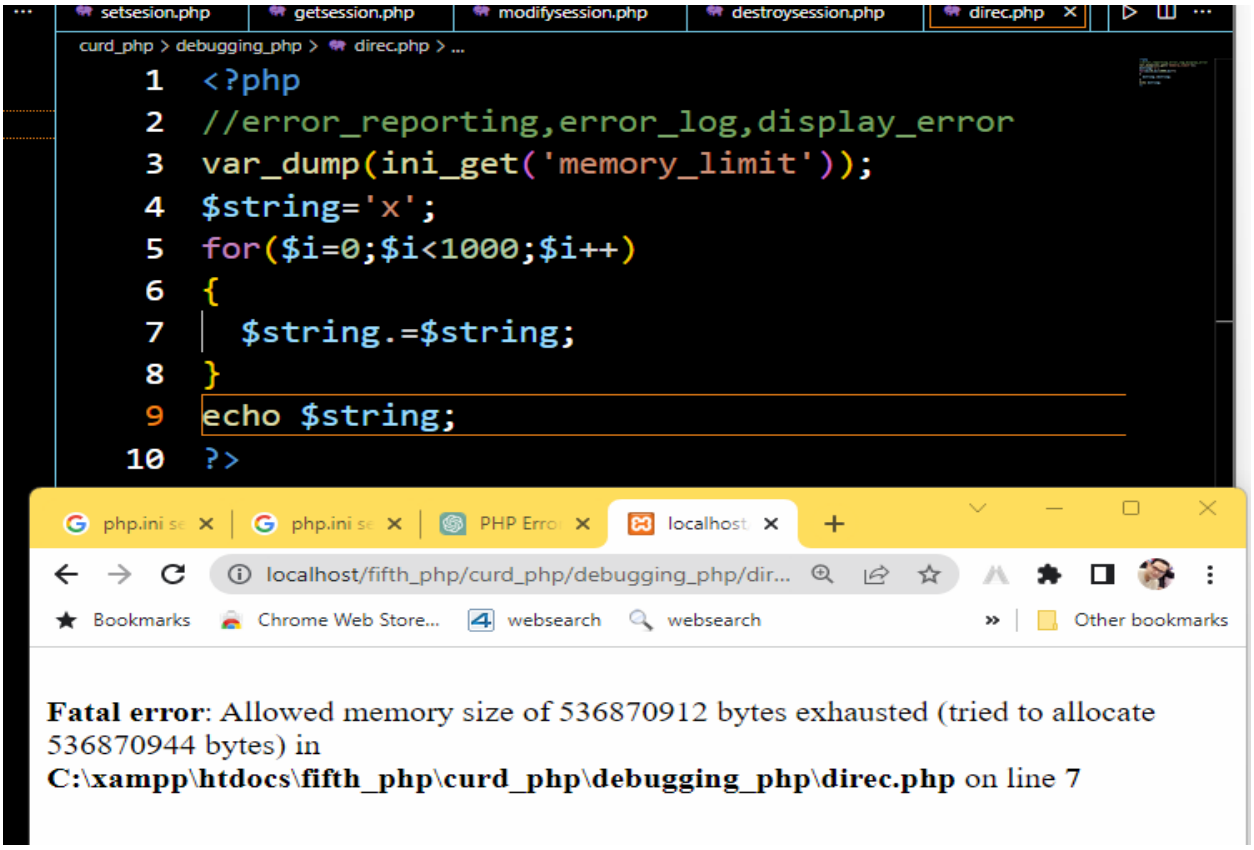
setsesion.php | getsession.php | modifysession.php | destroysession.php | direc.php

curd_php > debugging_php > direc.php

```php
<?php
//error_reporting,error_log,display_error
var_dump(ini_get('memory_limit'));
?>
```

localhost/fifth_php/curd_php/debugging_php/direc.php

string(4) "512M"



Types of errors in PHP

Basically, an error is a mistake in a program that may be caused by writing incorrect syntax or incorrect code. An error message is displayed on your browser containing the filename along with location, a message describing the error, and the line number in which error has occurred.

There are usually different types of error. In PHP, mainly four types of errors are considered:

1. Syntax Error or Parse Error
2. Fatal Error
3. Warning Error

We will discuss all these errors in detail with examples:

1.Syntax Error or Parse Error

A syntax error is a mistake in the syntax of source code, which can be done by programmers due to their lack of concern or knowledge. It is also known as **Parse error**. Compiler is used to catch the syntax error at compile time.

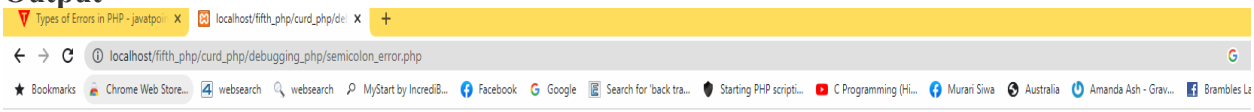
Note: Syntax error stop the execution of the code.

These errors can occur due to these common reasons like unclosed quotes, missing semicolon, extra or missing parentheses, or unclosed brackets and many more. While compiling the program, syntax error can be caught by the compiler. It gives a parse error or syntax error message.

Example 1: Missing semicolon

```
<?php
    /*------------------syntax error-------
------------*/
    echo "Alex: Hie! I'm Alex. </br>";
    echo "Bob: I'm Bob. How are you?"
    echo "Alex: I'm good! and you?";
    echo "Bob: I'm also good";
?>
```

Output



Parse error: syntax error, unexpected token "echo", expecting "," or ";" in **C:\xampp\htdocs\fifth_php\curd_php\debugging_php\semicolon_error.php** on line **5**

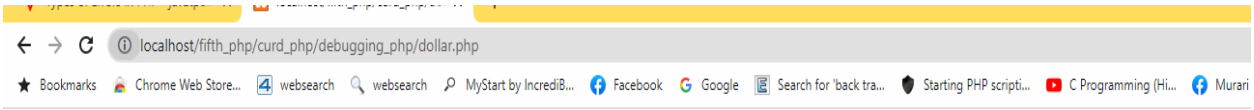
Explanation: In this above example, a semicolon (;) was missing in **line 5**. So, it generated a parse error and displayed an error message on browser as given in the output.

Example 2: Missing dollar symbol

```
?php
    /*------------------syntax error-------
------------*/
```

```
    $telecom = "Airtel";
    automobile = "Jaguar";
    echo $telecom;
    echo $automobile;
?>
```

Output



Parse error: syntax error, unexpected token "=" in **C:\xampp\htdocs\fifth_php\curd_php\debugging_php\dollar.php** on line **4**

Explanation: In this above example, dollar ($) symbol was missing in **line 5**. So, it generated a parse error and displayed an error message on browser as given in the output.

2.Fatal Error

A fatal error is another type of error, which is occurred due to the use of undefined function. The PHP compiler understands the PHP code but also recognizes the undefined function. This means that when a function is called without providing its definition, the PHP compiler generates a fatal error.

A fatal error is generated when a function is called without its definition. See the below example containing the fatal error -

Example: Calling undefined function

```
<?php
    /*-----------------fatal error--------
----------*/
    function add($f1, $f2) {
        $sum = $f1 + $f2;
        echo "Addition:" . $sum;
```

```
    }

    $f1 = 23;
    $f2 = 56;

    //call the function that is not defined

    //Generate fatal error
    catch_fatal_error();
    //echo "Fatal Error";
?>
```

In the above code we have defined the add() function but called other function, which is **catch_fatal_error()**. Therefore, it generates a fatal error and print an error message on the browser as given below:

Output

Fatal error: Uncaught Error: Call to undefined function catch_fatal_error() in C:\xampp\htdocs\fifth_php\curd_php\debugging_php\fatal_error.php:13 Stack trace: #0 {main} thrown in **C:\xampp\htdocs\fifth_php\curd_php\debugging_php\fatal_error.php** on line **13**

3.Warning Error

A warning is generated when the programmer tries to include a missing file. The PHP function calls that missing file which does not exist. The warning error does not stop/prevent the execution of the program.

The main reason behind generating a warning error is to pass an incorrect number of parameters to a function or to include a missing file.

Example: Include missing file

```php
<?php
    /*-------------------warning error-----
-------------*/
    $cmpny = 'php';
    echo "Warning Error: ";

    //include a file in the code
    include ('jtp.php');
?>
```

Output

Explanation: In this example, we tried to include a file in our program, which does not exist. So, it generated a warning and displayed an error message.

Error Handling in php

In PHP, error handling is the process of managing and dealing with errors that occur during script execution. Error handling helps you identify and fix errors in your PHP code, making your scripts more robust and reliable.

PHP provides several built-in functions for error handling, including:

1.error_reporting: This function sets the error reporting level for your script. You can specify which errors you want to see by using different levels of error reporting.

2.ini_set: This function allows you to set PHP configuration options at runtime. You can use this function to change the error reporting level, display errors or suppress errors.

3.set_error_handler: This function allows you to set a custom error handler function. This function is called whenever an error occurs, and you can use it to handle errors in a way that is specific to your application.

4.trigger_error: This function allows you to trigger a user-defined error. You can use this function to handle errors in your code.

5.try/catch: This is a mechanism for handling exceptions, which are a type of error that occurs when a problem arises in your code that cannot be handled by normal error handling mechanisms.

When creating scripts and web applications, error handling is an important part. If your code lacks error checking code, your program may look very unprofessional and you may be open to security risks.

This tutorial contains some of the most common error checking methods in PHP.

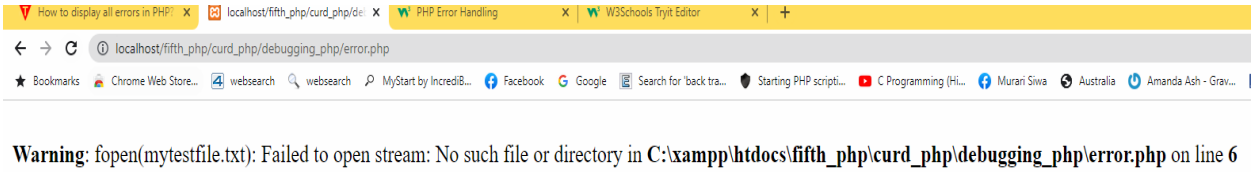
We will show different error handling methods:

- Simple "die()" statements
- Custom errors and error triggers
- Error reporting

Basic Error Handling: Using the die() function

```
<!DOCTYPE html>
<html>
<body>

<?php
$file=fopen("mytestfile.txt","r");
?>

</body>
```

```
</html>
```

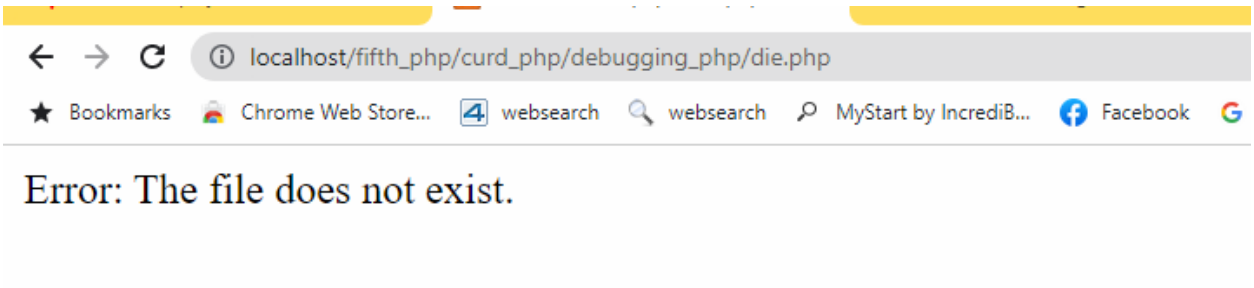


Warning: fopen(mytestfile.txt): Failed to open stream: No such file or directory in **C:\xampp\htdocs\fifth_php\curd_php\debugging_php\error.php** on line **6**

To prevent the user from getting an error message like the one above, we test whether the file exist before we try to access it:

Example:

```
<!DOCTYPE html>
<html>
<body>

<?php
if(file_exists("mytestfile.txt")) {
  $file = fopen("mytestfile.txt", "r");
} else {
  die("Error: The file does not exist.");
}
?>

</body>
</html>
```



The code above is more efficient than the earlier code, because it uses a simple error handling mechanism to stop the script after the error.

However, simply stopping the script is not always the right way to go. Let's take a look at alternative PHP functions for handling errors.

2.Creating a Custom Error Handler

Creating a custom error handler is quite simple. We simply create a special function that can be called when an error occurs in PHP.

This function must be able to handle a minimum of two parameters (error level and error message) but can accept up to five parameters (optionally: file, line-number, and the error context):

Syntax:error_function(error_level,error_message,error_file,error_line,error_context)

Parameter	Description
error_level	Required. Specifies the error report level for the user-defined error. Must be a value number. See table below for possible error report levels
error_message	Required. Specifies the error message for the user-defined error
error_file	Optional. Specifies the filename in which the error occurred
error_line	Optional. Specifies the line number in which the error occurred
error_context	Optional. Specifies an array containing every variable, and their values, in use when the error occurred

Error Report levels

These error report levels are the different types of error the user-defined error handler can be used for:

Value	Constant	Description
1	E_ERROR	A fatal run-time error. Execution of the script is stopped
2	E_WARNING	A non-fatal run-time error. Execution of the script is not stopped
8	E_NOTICE	A run-time notice. The script found something that might be an error, but could also happen when running a script normally
256	E_USER_ERROR	A fatal user-generated error. This is like an E_ERROR, except it is generated by the PHP script using the function trigger_error()
512	E_USER_WARNING	A non-fatal user-generated warning. This is like an E_WARNING, except it is generated by the PHP script using the function trigger_error()
1024	E_USER_NOTICE	A user-generated notice. This is like an E_NOTICE, except it is generated by the PHP script using the function trigger_error()
2048	E_STRICT	Not strictly an error.
8191	E_ALL	All errors and warnings (E_STRICT became a part of E_ALL in PHP 5.4)

Now lets create a function to handle errors:

```
function customError($errno, $errstr)
{
  echo "<b>Error:</b> [$errno] $errstr<br>";
  echo "Ending Script";
  die();
}
```

The code above is a simple error handling function. When it is triggered, it gets the error level and an error message. It then outputs the error level and message and terminates the script.

Now that we have created an error handling function we need to decide when it should be triggered.

error_reporting

The error_reporting() function specifies which errors are reported.

PHP has many levels of errors, and using this function sets that level for the current script.

Syntax

error_reporting(*level*);

Parameter Values

Parameter	Description
level	Optional. Specifies the error-report level for the current script. Error numbers and named constants are accepted. **Note:** Named constants are recommended to ensure compatibility for future PHP versions

Technical Details

Return Value:	Returns the old error reporting level or the current error reporting level if no *level* parameter is given
PHP Version:	4.0+
PHP Changelog:	PHP 5.4: E_STRICT is now a part of E_ALL. PHP 5.3: New: E_DEPRECATED and E_USER_DEPRECATED. PHP 5.2: New: E_RECOVERABLE_ERROR. PHP 5.0: New: E_STRICT.

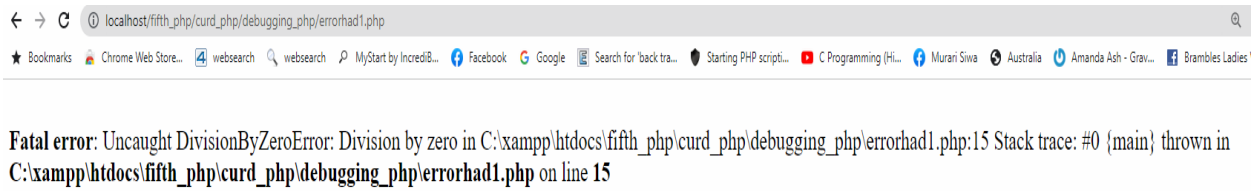
```php
<?php
// Turn off error reporting
error_reporting(0);

// Report runtime errors
error_reporting(E_ERROR | E_WARNING |
E_PARSE);

// Report all errors
error_reporting(E_ALL);

// Same as error_reporting(E_ALL);
ini_set("error_reporting", E_ALL);

// Report all errors except E_NOTICE
error_reporting(E_ALL & ~E_NOTICE);
?>
```

Example:

```php
<?php
// Turn on error reporting
error_reporting(E_ALL);

// Set a custom error handler function
```

```
function myErrorHandler($errno, $errstr,
$errfile, $errline)
{
    echo "<b>Error:</b> [$errno]
$errstr<br>";
    echo "  Error on line $errline in
$errfile<br>";
}

set_error_handler("myErrorHandler");

// Trigger an error
$var = 1 / 0;

// Exception handling
try {
    // Code that may throw an exception
    throw new Exception("An error
occurred");
} catch (Exception $e) {
    // Handle the exception
    echo "Caught exception: " . $e-
>getMessage();
}
```

```
?>
```



Fatal error: Uncaught DivisionByZeroError: Division by zero in C:\xampp\htdocs\fifth_php\curd_php\debugging_php\errorhad1.php:15 Stack trace: #0 {main} thrown in **C:\xampp\htdocs\fifth_php\curd_php\debugging_php\errorhad1.php** on line **15**

In the example above, we turn on error reporting, set a custom error handler function, trigger an error, and handle an exception using try/catch block. These are just a few of the many ways to handle errors in PHP.

Exception

An exception is an object that describes an error or unexpected behaviour of a PHP script.

- Exceptions are thrown by many PHP functions and classes.
- User defined functions and classes can also throw exceptions.
- Exceptions are a good way to stop a function when it comes across data that it cannot use.

Throwing an Exception

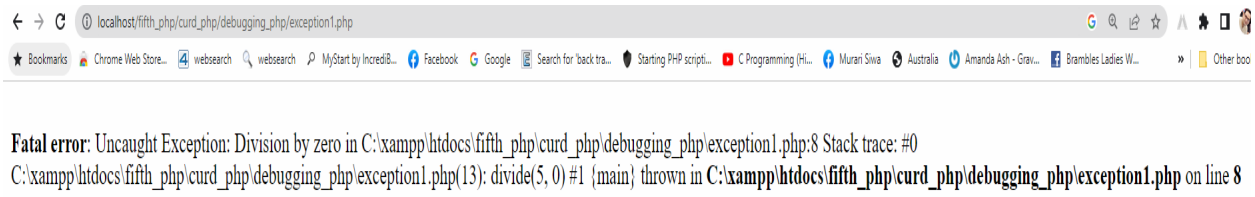
The throw statement allows a user defined function or method to throw an exception. When an exception is thrown, the code following it will not be executed.

- If an exception is not caught, a fatal error will occur with an "Uncaught Exception" message.

Lets try to throw an exception without catching it:

```
     <!DOCTYPE html>

<html>
<body>

<?php
function divide($dividend, $divisor) {
  if($divisor == 0) {
    throw new Exception("Division by
zero");
```

```
  }
  return $dividend / $divisor;
}

echo divide(5, 0);
?>

</body>
</html>
```



Fatal error: Uncaught Exception: Division by zero in C:\xampp\htdocs\fifth_php\curd_php\debugging_php\exception1.php:8 Stack trace: #0 C:\xampp\htdocs\fifth_php\curd_php\debugging_php\exception1.php(13): divide(5, 0) #1 {main} thrown in **C:\xampp\htdocs\fifth_php\curd_php\debugging_php\exception1.php** on line **8**

The try...catch Statement

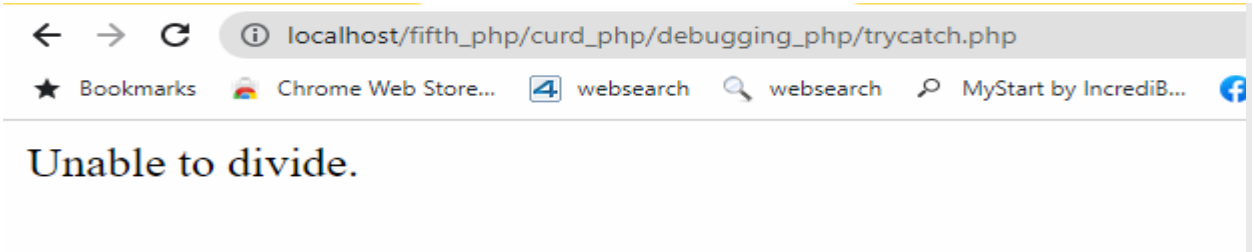
To avoid the error from the example above, we can use the try...catch statement to catch exceptions and continue the process.

Syntax

```
try {
  code that can throw exceptions
} catch(Exception $e) {
  code that runs when an exception is caught
}
```

```
      <!DOCTYPE html>

<html>
<body>

<?php
```

```
function divide($dividend, $divisor) {
  if($divisor == 0) {
    throw new Exception("Division by
zero");
  }
  return $dividend / $divisor;
}

try {
  echo divide(5, 0);
} catch(Exception $e) {
  echo "Unable to divide.";
}
?>

</body>
</html>
```



The catch block indicates what type of exception should be caught and the name of the variable which can be used to access the exception. In the example above, the type of exception is Exception and the variable name is $e.

The try...catch...finally Statement

The try...catch...finally statement can be used to catch exceptions. Code in the finally block will always run regardless of whether an exception was caught. If finally is present, the catch block is optional.

Syntax

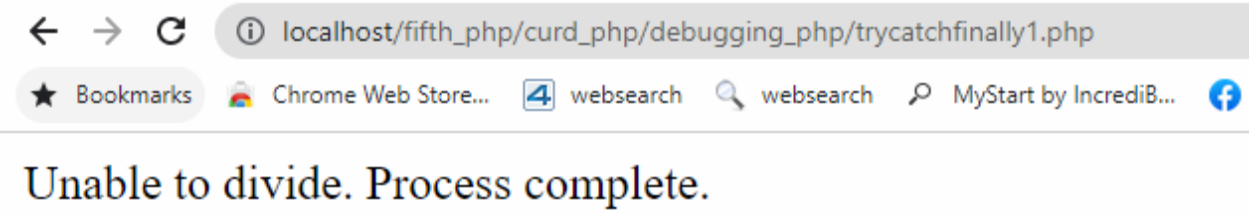
```
try {
  code that can throw exceptions
} catch(Exception $e) {
  code that runs when an exception is caught
} finally {
  code that always runs regardless of whether an exception was caught
}
```

Example:

Show a message when an exception is thrown and then indicate that the process has ended:

```
<!DOCTYPE html>
<html>
<body>

<?php
function divide($dividend, $divisor) {
  if($divisor == 0) {
    throw new Exception("Division by
zero");
  }
  return $dividend / $divisor;
}

try {
  echo divide(5, 0);
```

```
} catch(Exception $e) {
  echo "Unable to divide. ";
} finally {
  echo "Process complete.";
}
?>

</body>
</html>
```



Example:

Output a string even if an exception was not caught:

```
<?php
function divide($dividend, $divisor) {
  if($divisor == 0) {
    throw new Exception("Division by
zero");
  }
  return $dividend / $divisor;
}
```

```php
try {
  echo divide(5, 0);
} finally {
  echo "Process complete.";
}
?>
```

localhost/fifth_php/curd_php/debugging_php/tryfinally.php

Process complete.
Fatal error: Uncaught Exception: Division by zero in C:\xampp\htdocs\fifth_php\curd_php\debugging_php\tryfinally.php:4 Stack trace: #0 C:\xampp\htdocs\fifth_php\curd_php\debugging_php\tryfinally.php(10): divide(5, 0) #1 {main} thrown in **C:\xampp\htdocs\fifth_php\curd_php\debugging_php\tryfinally.php** on line **4**

The Exception Object

The Exception Object contains information about the error or unexpected behaviour that the function encountered.

Syntax:
new Exception(message, code, previous)

Parameter Values

Parameter	Description
message	Optional. A string describing why the exception was thrown
code	Optional. An integer that can be used used to easily distinguish this exception from others of the same type
previous	Optional. If this exception was thrown in a catch block of another exception, it is recommended to pass that exception into this parameter

Methods

When catching an exception, the following table shows some of the methods that can be used to get information about the exception:

Method	Description
getMessage()	Returns a string describing why the exception was thrown
getPrevious()	If this exception was triggered by another one, this method returns the previous exception. If not, then it returns *null*
getCode()	Returns the exception code
getFile()	Returns the full path of the file in which the exception was thrown
getLine()	Returns the line number of the line of code which threw the exception

Example:

Output information about an exception that was thrown:

```
<!DOCTYPE html>
<html>
<body>

<?php
function divide($dividend, $divisor) {
  if($divisor == 0) {
    throw new Exception("Division by zero",
1);
  }
```

```
  return $dividend / $divisor;
}

try {
  echo divide(5, 0);
} catch(Exception $ex) {
  $code = $ex->getCode();
  $message = $ex->getMessage();
  $file = $ex->getFile();
  $line = $ex->getLine();
  echo "Exception thrown in $file on line
$line: [Code $code]
  $message";
}
?>

</body>
</html>
```

Exception thrown in C:\xampp\htdocs\fifth_php\curd_php\debugging_php\exceptionobject.php on line 8: [Code 1] Division by zero

Error Suppression in php

In PHP, error suppression is the process of preventing PHP from displaying error messages when a code block encounters an error. This can be useful in situations where you don't want the user to see error messages or where the error is not critical to the functionality of the script.

The most common way to suppress errors in PHP is to use the @ symbol before the function or expression that might generate an error. For example:

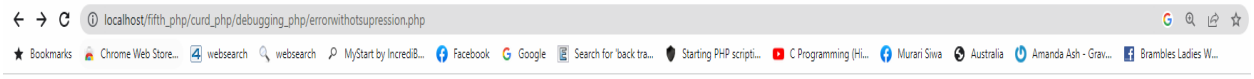
$result = @mysql_query($query);

In this example, if there is an error in the mysql_query function, PHP will not display the error message. Instead, the $result variable will be assigned the value false.

While error suppression can be useful in some situations, it is generally not recommended to use it as a way to handle errors. Suppressing errors can make it more difficult to debug and fix issues in your code, and it can also create security vulnerabilities if error messages contain sensitive information. Instead, it is best to use proper error handling techniques, such as try-catch blocks, to handle errors in your code.

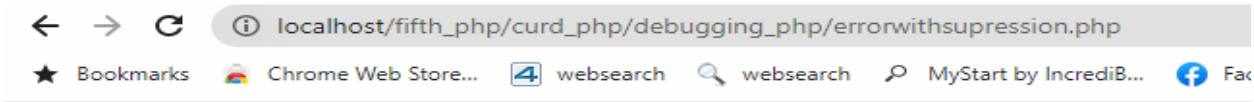
Following code tries to open a non-existing file for read operation, but PHP parser reports warning:

```
<?php
$fp=fopen("nosuchfile.txt","r");
echo "Hello World
";
?>
```



Warning: fopen(nosuchfile.txt): Failed to open stream: No such file or directory in **C:\xampp\htdocs\fifth_php\curd_php\debugging_php\errorwithotsupression.php** on line **2**
Hello World

```
<?php
$fp=@fopen("nosuchfile.txt","r");
echo "Hello World";
?>
```



Hello World

Set Error Handler

The default error handler for PHP is the built in error handler. We are going to make the function above the default error handler for the duration of the script.

It is possible to change the error handler to apply for only some errors, that way the script can handle different errors in different ways. However, in this example we are going to use our custom error handler for all errors:
set_error_handler("customError");

Since we want our custom function to handle all errors, the set_error_handler() only needed one parameter, a second parameter could be added to specify an error level.

Example

Testing the error handler by trying to output variable that does not exist:

```php
<?php
//error handler function
function customError($errno, $errstr) {
  echo "<b>Error:</b> [$errno] $errstr";
}

//set error handler
set_error_handler("customError");

//trigger error
echo($test);
?>
```

Trigger an Error

In a script where users can input data it is useful to trigger errors when an illegal input occurs. In PHP, this is done by the trigger_error() function.

Example

In this example an error occurs if the "test" variable is bigger than "1":

```
<?php
$test=2;
if ($test>=1) {
  trigger_error("Value must be 1 or
below");
}
?>
```

Notice: Value must be 1 or below in **C:\xampp\htdocs\fifth_php\curd_php\debugging_php\trigger_php.php** on line **4**

An error can be triggered anywhere you wish in a script, and by adding a second parameter, you can specify what error level is triggered.

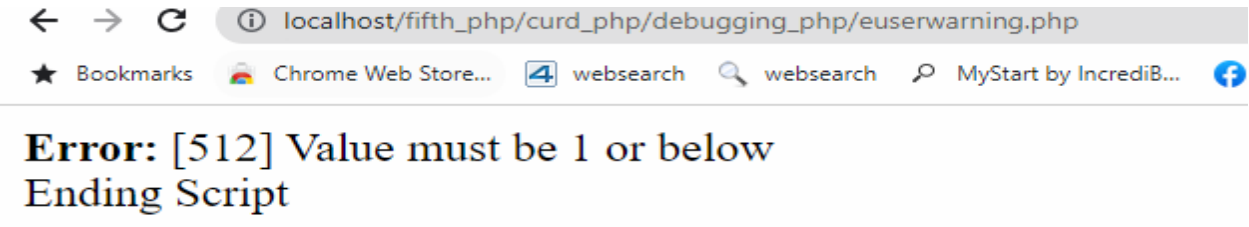
Possible error types:

- E_USER_ERROR - Fatal user-generated run-time error. Errors that can not be recovered from. Execution of the script is halted
- E_USER_WARNING - Non-fatal user-generated run-time warning. Execution of the script is not halted
- E_USER_NOTICE - Default. User-generated run-time notice. The script found something that might be an error, but could also happen when running a script normally

Example

In this example an E_USER_WARNING occurs if the "test" variable is bigger than "1". If an E_USER_WARNING occurs we will use our custom error handler and end the script:

```php
<?php
//error handler function
function customError($errno, $errstr) {
  echo "<b>Error:</b> [$errno]
$errstr<br>";
  echo "Ending Script";
  die();
}

//set error handler
set_error_handler("customError",E_USER_WARN
ING);

//trigger error
$test=2;
if ($test>=1) {
  trigger_error("Value must be 1 or
below",E_USER_WARNING);
}
?>
```



Now that we have learned to create our own errors and how to trigger them, lets take a look at error logging.

Error Logging

By default, PHP sends an error log to the server's logging system or a file, depending on how the error_log configuration is set in the php.ini file. By using the error_log() function you can send error logs to a specified file or a remote destination.

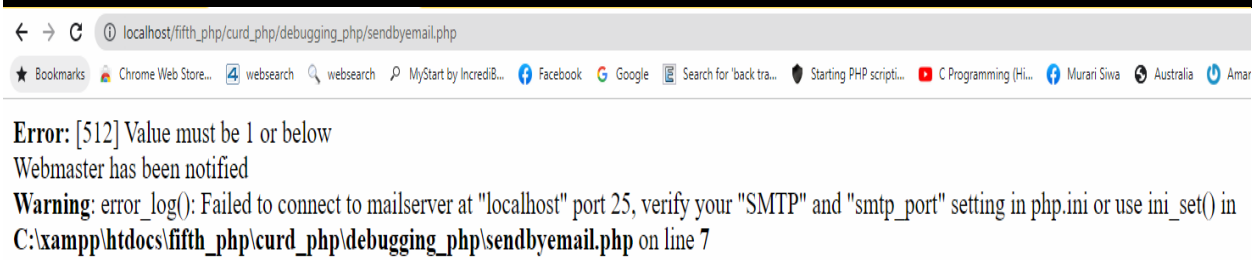
Sending error messages to yourself by e-mail can be a good way of getting notified of specific errors.

Send an Error Message by E-Mail

In the example below we will send an e-mail with an error message and end the script, if a specific error occurs:

```
<?php
//error handler function
function customError($errno, $errstr) {
  echo "<b>Error:</b> [$errno]
$errstr<br>";
  echo "Webmaster has been notified";
  error_log("Error: [$errno] $errstr",1,
  "someone@example.com","From:
webmaster@example.com");
}

//set error handler
```

```
set_error_handler("customError",E_USER_WARNING);

//trigger error
$test=2;
if ($test>=1) {
  trigger_error("Value must be 1 or below",E_USER_WARNING);
}
?>
```



This should not be used with all errors. Regular errors should be logged on the server using the default PHP logging system.